TorchDCM: An Open, Differentiable, and Benchmarkable Framework for Discrete Choice Estimation

Baichuan Mo

July 11, 2026

Abstract

Discrete choice models are widely used in transportation, marketing, operations, and service-system research. Researchers, however, often need to combine mature econometric model classes with large-scale tensor computation, automatic differentiation, GPU execution, and reproducible estimator validation. Existing software addresses these requirements only in a fragmented manner. Biogeme, Apollo, and established R packages provide rich econometric functionality, while GPU-oriented choice software and general tensor frameworks do not always expose the same model specification, inference, and post-estimation workflow.

We introduce TorchDCM, an open Python package that places discrete-choice abstractions on top of a PyTorch likelihood engine. The package supports multinomial, nested, cross-nested, mixed, WTP-space mixed, scaled, ordered, latent-class, error-components, and hybrid choice models. Its common design supports ragged choice sets, availability masks, panel likelihoods, simulation draws, automatic differentiation, Hessian-based inference, robust covariance for supported models, WTP, elasticities, and probability prediction. The package is accompanied by a validation layer that aligns public data, specifications, initial values, simulation draws, covariance definitions, and runtime accounting against Biogeme, Apollo, SciPy, `mlogit`, `gmnl`, and `xlogit`.

The contribution of TorchDCM is software infrastructure rather than a new random utility theory. The package connects econometric reporting with a tensor-native execution model, while the benchmark system makes numerical agreement and computational scaling explicit. Current experiments establish full-estimation parity for several standard and structured model families and show favorable scaling in selected large synthetic cases. The results position TorchDCM as an open platform for researchers who need differentiable choice models, econometric inference, and transparent cross-estimator validation.

1 Introduction

Discrete choice models are a central tool for representing decisions among mutually exclusive alternatives. They are used to study travel mode choice, product and brand selection, technology adoption, service choice, and many other operations and policy problems. The methodological requirements of contemporary applications extend beyond evaluating a multinomial logit likelihood. A practical estimation system must support alternative availability, unequal choice-set sizes, repeated choices, random coefficients, latent classes, latent variables, simulation-based likelihoods, statistical inference, and economically meaningful post-estimation quantities such as willingness to pay and elasticities.

At the same time, empirical choice problems are becoming larger and more computationally demanding. Researchers increasingly work with large surveys, high-dimensional attributes, many simulation draws, and models that must be embedded in differentiable pipelines. These settings

create a tension between traditional econometric software and modern tensor computation. A package may provide a mature model and inference interface without exposing a unified GPU execution path, or it may provide fast automatic differentiation without the discrete-choice data structures and econometric reporting expected by applied researchers.

1.1 The software gap

The existing software ecosystem is valuable but fragmented along three related dimensions. First, packages use different computational abstractions. Biogeme provides a mature Python environment with symbolic model specification and compiled numerical backends, while Apollo provides a flexible choice-modeling framework in R. These designs are effective for conventional applications, but neither is organized around a common PyTorch tensor and device-execution model. Consequently, model construction, compilation, interpreter, and data-conversion overheads can become material as the sample size, number of alternatives, model dimension, or number of simulation draws increases. This is not a claim that these packages are universally slower; their performance depends on model structure, initialization, backend, and compilation overhead. Instead, it identifies a software-design gap: large-scale differentiable choice estimation is not the primary organizing principle of the traditional interfaces.

Second, model specification and data handling are distributed across languages and package-specific conventions. Long-format and wide-format data, availability rules, alternative coding, fixed-parameter conventions, scale normalizations, and panel identifiers must often be translated before the same model can be estimated in another package. Simulation draws and covariance definitions create additional alignment problems for mixed and latent-variable models.

Third, estimation, inference, and validation are often treated as separate workflows. A package may estimate parameters without providing the same Hessian, robust covariance, WTP, elasticity, and probability objects needed to audit an implementation. Cross-package comparisons then become ad hoc rather than reproducible experiments. TorchDCM addresses this fragmentation by making the data representation, specification layer, likelihood kernel, inference objects, and benchmark protocol explicit parts of one software system.

1.2 TorchDCM and research questions

TorchDCM is a PyTorch-first package for discrete choice model estimation and econometric inference. Its central design is to represent choice data and model specifications in discrete-choice-native objects, compile them into tensor operations, and use the same execution path for likelihood evaluation, optimization, prediction, simulation, and Hessian-based inference. A separate validation layer connects the package to public datasets and external reference estimators without making those heavyweight dependencies part of the installable package.

The paper addresses three questions:

1. Can a common PyTorch-native architecture support both standard and advanced discrete choice models while retaining an econometric estimation interface?

2. Do TorchDCM estimates, probabilities, covariance quantities, and post-estimation measures agree with mature reference implementations under aligned specifications?

3. How does the tensor execution path behave as the sample size, number of alternatives, model dimension, and number of simulation draws increase?

1.3 Contributions

This paper makes three software contributions. First, it introduces an open PyTorch-first package that combines a model specification language, choice-data containers, estimation routines, econometric inference, and post-estimation outputs. Second, it documents a common likelihood architecture for ragged choice sets, panel likelihoods, simulation draws, constrained parameters, and automatic differentiation. The architecture supports a model zoo that includes MNL, nested and cross-nested logit, mixed logit, WTP-space mixed logit, scaled models, ordered responses, latent classes, error components, and hybrid choice components. Third, it develops a reproducible benchmark system that separates full estimation, fixed-parameter likelihood replay, and fit-then-replay comparisons, and evaluates numerical agreement together with runtime and scaling.

The paper does not claim that these established econometric model families are new theories. The intended contribution is an extensible and auditable implementation that makes advanced models compatible with tensor computation and systematic estimator validation. The current evidence is strongest for full-estimation MNL, nested-logit-family, and ordered-response cases, with additional mixed-logit and latent-class evidence under explicitly documented shared-draw or replay protocols.

The remainder of the paper is organized as follows. Section 2 positions TorchDCM relative to existing choice-modeling software. Section 3 describes the package architecture and user-facing objects. Section 4 presents the likelihood, simulation, inference, and device-execution design. Section 5 defines the public-data benchmark and alignment protocol. Section 6 reports numerical parity and computational scaling. Sections 7 and 8 discuss limitations and future directions.

2 Related Software

2.1 Econometric choice-modeling packages

Biogeme is a widely used Python environment for specifying and estimating discrete choice models, with a mature model syntax and econometric reporting workflow [2]. Apollo is a flexible R package designed for a broad range of choice models, including mixed logit, latent class, and hybrid-choice structures [4]. The R packages `mlogit` and `gmnl` provide established implementations of random utility models and heterogeneous preference structures [3, 6]. These packages are important reference implementations for TorchDCM; the goal is not to replace their econometric maturity.

Their computational and user-facing interfaces, however, are not identical. They differ in programming language, data format, parameter transformations, simulation-draw conventions, optimization backend, and covariance reporting. These differences are manageable for a single application but become costly when researchers want to move between models, compare estimators, or combine choice estimation with differentiable computational workflows. Runtime can also be affected by symbolic model construction, compilation, interpreter overhead, or cross-language data conversion. The benchmark in Section 6 therefore reports both numerical agreement and runtime, rather than treating runtime as an isolated claim about an entire software package.

2.2 Tensor and GPU-oriented choice software

The Python ecosystem also contains packages that emphasize computational performance. In particular, `xlogit` provides GPU-accelerated estimation for mixed logit models [1]. General-purpose PyTorch implementations provide automatic differentiation and flexible tensor operations [5], but they do not automatically provide choice-specific data structures, econometric covariance objects, or a common benchmark protocol. TorchDCM is positioned between these two traditions: it uses

PyTorch as the computational backend while exposing named parameters, utility specifications, panel data, model-specific transformations, and econometric result objects.

2.3 Positioning of TorchDCM

TorchDCM differs from a single-model GPU implementation in four ways. It provides a common API across model families; it treats inference and post-estimation as first-class outputs; it supports ragged and panel choice data within the same tensor representation; and it pairs the package with an external validation layer. The resulting contribution is software infrastructure: researchers can inspect, extend, and benchmark individual likelihood modules without rebuilding the entire data, optimization, and inference workflow.

3 Package Design

TorchDCM is organized into a lightweight package layer and a separate validation layer. The package layer contains data containers, specification objects, model classes, estimation routines, inference utilities, and result objects. The validation layer contains public data processing, reference estimator wrappers, benchmark scripts, and generated reports. This separation keeps the user-facing package installable while allowing the benchmark system to depend on Biogeme, Apollo, R packages, and larger public datasets.

3.1 Choice data and panel structure

The central data object is `ChoiceDataset`. It stores the row-pointer representation of observations, alternative identifiers, chosen rows, alternative-level variables, availability indicators, observation weights, and optional observation-level variables. The row-pointer representation supports both balanced choice sets and ragged choice sets in which different observations have different feasible alternatives. Wide transportation tables can be converted into this representation through the same constructor used for long data.

Optional individual identifiers define a `PanelStructure`. This mapping allows repeated choices to be aggregated by individual and allows cluster codes to be aligned with the likelihood contributions. The representation is shared by standard models and simulation-based models, so panel likelihood behavior is not hidden inside a model-specific data loader.

3.2 Specification and model objects

Users define named `Beta` parameters and combine them with variables in a `UtilitySpec`. The same expression representation is used by MNL and extended by nests, random coefficients, WTP coefficients, latent classes, scale equations, thresholds, and latent-variable components. Fixed and free parameters are tracked during compilation, which makes identification and parameter ordering explicit.

The model layer includes multinomial, nested, and cross-nested logit models; mixed and WTP-space mixed logit; error-components logit; latent-class logit; alternative-specific and covariate-scaled MNL; ordered logit and probit; and a hybrid choice model with normally distributed latent variables and continuous Gaussian indicators. These models share the same result and prediction interface even when their likelihood kernels use different integration or parameter-transformation steps.

3.3 Econometric result objects

A fitted model returns a `ChoiceResults` object rather than only a parameter tensor. The object stores estimates, likelihood values, gradients, observed information, covariance matrices, convergence diagnostics, standard errors, test statistics, confidence intervals, probabilities, predictions, WTP, and elasticities when defined by the model. For supported models, robust and cluster covariance matrices are constructed from observation-level scores. This design makes downstream econometric auditing part of the estimator workflow.

4 Implementation

4.1 Compiled tensor likelihoods

Each model maps a data object and parameter vector to utilities, probabilities, log-likelihood contributions, and result objects. The specification compiler converts alternative-specific utility expressions into free-parameter and fixed-parameter design matrices. Linear utility evaluation then uses tensor matrix operations, while fixed terms are added through a separate fixed design. Compiled objects are cached for a data/model pair so that repeated likelihood, gradient, prediction, and Hessian evaluations reuse the same data-dependent representation.

For balanced choice sets, utilities are reshaped into observation-by-alternative blocks. For ragged choice sets, TorchDCM uses row-to-observation mappings and scatter operations. Availability is applied before normalization, and stable `logsumexp` calculations avoid unnecessary overflow or underflow. Thus, unavailable alternatives receive zero probability while the feasible set is normalized separately for every observation.

4.2 Automatic differentiation and inference

Unconstrained parameters are optimized directly with a PyTorch LBFGS routine. Parameters with positivity or interval restrictions are optimized in an unconstrained internal representation and mapped to the natural scale inside the likelihood. After estimation, automatic differentiation computes the score and Hessian. The observed information matrix and its pseudoinverse provide the classic covariance estimate where the model supports it. For models with observation-level scores, the same result object can construct robust and cluster covariance through the sandwich formula.

This implementation makes inference part of the same differentiable graph as the likelihood. It also exposes the transformation Jacobian used to map covariance matrices from internal to natural parameter scales. This detail is important for dissimilarity parameters, random-coefficient standard deviations, scale parameters, and ordered-response thresholds.

4.3 Simulation and panel likelihoods

Mixed logit, WTP-space mixed logit, and hybrid choice models use explicit draw tensors. Draws can be generated from a seed, supplied by the user, or made antithetic for variance reduction. Random coefficients support normal, lognormal, and negative-lognormal transformations. Correlated random effects are represented with a Cholesky factor whose diagonal is constrained through positive standard deviations.

For panel data, the same individual-level draw is used across repeated choices. Observation-level log probabilities are first summed within an individual and then averaged over draws in log

space. This order of operations preserves the panel likelihood rather than treating repeated choices as independent cross-sectional observations.

4.4 Advanced model components

The most distinctive model components are implemented as extensions of the common likelihood architecture. WTP-space mixed logit estimates economically interpretable WTP coefficients directly instead of obtaining them only as a post-estimation ratio. The hybrid choice model jointly evaluates the choice likelihood and Gaussian measurement likelihood and integrates over latent variables using Monte Carlo draws. Error components are exposed through a convenience specification that augments a mixed-logit utility with alternative-specific random loadings. Latent-class models combine class-specific utilities with a covariate-dependent membership model and expose posterior class probabilities.

4.5 Device execution

All model classes accept a PyTorch device and move compiled data, simulation draws, likelihood calculations, predictions, and covariance calculations to that device. The default dtype is float64 because the target use case is econometric estimation and Hessian-based inference. CPU and CUDA use the same model code path, which allows numerical comparisons under identical initialization and simulation draws. The benchmark therefore separates cross-package comparisons from within-TorchDCM CPU/GPU scaling experiments.

5 Benchmark Data and Protocol

The validation system is designed to test a software system rather than only a single likelihood value. It uses public datasets, explicit model specifications, and executable wrappers for external estimators. Public examples from Biogeme, Apollo, and R choice-modeling packages are preferred because their data and model conventions can be documented and independently inspected.

5.1 Alignment rules

Every benchmark case records the data source, alternative ordering, availability rules, variable scaling, parameter names, fixed-parameter normalization, starting values, simulation-draw convention, covariance definition, device, and stopping criteria. These details are necessary because apparently small differences in alternative coding, cost scaling, draw ordering, or panel aggregation can produce different estimates even when two implementations are correct.

5.2 Benchmark modes

The experiments distinguish three modes. In *full estimation*, each backend estimates the model independently from aligned starting values. In *fixed-parameter replay*, all backends receive the same parameters and, for simulated models, the same draw matrix; this isolates likelihood and probability implementation differences from optimizer and simulation noise. In *fit-then-replay*, TorchDCM estimates the parameters and external packages evaluate the resulting specification at those fixed parameters. The paper reports these modes separately and does not treat replay as evidence of independent external estimation.

5.3 Evaluation metrics

The primary numerical metrics are final log likelihood, parameter differences, choice-probability differences, covariance differences, standard-error differences, WTP differences, and elasticity differences where defined. Runtime is separated into parameter estimation and covariance/Hessian calculation when the backend exposes both. Synthetic experiments vary sample size N, number of alternatives J, number of observed variables K, feature correlation, and simulation draws. Stress experiments use a prespecified wall-clock limit and label timeouts as computational outcomes rather than numerical failures.

The comparison is intentionally not framed as a universal ranking of software packages. Small models can be dominated by startup and compilation overhead, and different packages expose different optimization and device choices. The goal is to characterize the trade-off between econometric maturity, interface flexibility, numerical agreement, and tensor-based scalability.

6 Computational Results

The computational results use two complementary benchmark families. The first family uses public empirical datasets aligned with Biogeme, Apollo, and R package examples. The second family is a pure synthetic controlled benchmark that varies the number of samples, alternatives, variables, and feature correlations under a known MNL data-generating process. The empirical results are organized by model family: MNL in Table 1, nested-logit-family models in Table 2, and mixed-logit models in Table 3.

Table 1: Real-data MNL runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for each aligned estimator. A dash means that no aligned wrapper is included for that dataset-estimator pair; *Fail* means the wrapper was attempted but did not complete.

Data	N	TorchDCM	SciPy	Biogeme	Apollo	`mlogit`	`gmnl`	`xlogit`	Consistent?
Swissmetro	10719	0.028	2.328	1.187	1.825	0.692	*Fail*	0.029	Yes
NHTS 2022	27375	0.099	44.345	1.752	9.843	2.898	31.955	0.711	Yes
Airline itinerary	3609	0.025	3.103	1.195	1.199	0.585	0.787	0.011	Yes
Parking Spain	1576	0.023	1.873	1.174	1.129	0.557	0.698	0.006	Yes
Telephone service	434	0.022	0.241	1.181	1.094	0.544	*Fail*	0.004	Yes
LPMC London	81086	0.074	59.196	1.398	21.171	4.000	7.769	0.325	Yes
Catsup	2798	0.021	0.375	1.158	1.120	0.053	0.705	0.008	Yes
Cracker	3292	0.028	0.544	1.185	1.138	0.057	0.716	0.009	Yes
Electricity	4308	0.032	2.821	1.592	1.282	0.197	0.970	0.013	Yes
HC	250	0.020	0.048	1.365	1.045	0.017	0.667	0.002	Yes
Heating	900	0.019	0.140	1.016	1.055	0.025	0.667	0.003	Yes
Mode	453	0.017	0.201	0.952	1.026	0.016	0.663	0.002	Yes
NOx	632	0.020	0.238	2.426	1.132	0.041	*Fail*	0.004	Yes
RiskyTransport	1793	0.038	1.147	1.815	1.214	0.045	*Fail*	*Fail*	Yes
Train	2929	0.032	1.771	0.886	1.137	0.033	0.705	0.005	Yes
Fishing	1182	0.033	0.813	1.176	1.101	1.176	0.707	0.006	Yes
ModeCanada	4324	0.028	1.960	1.155	1.200	0.613	*Fail*	*Fail*	Yes

Table 1 reports the main real-data MNL benchmark. Each row uses an actual public dataset and an aligned model specification. The columns report total remote wall-clock runtime for TorchDCM and every available benchmark estimator; the final column reports whether all successful aligned external estimators agree with TorchDCM under the prespecified tolerance rule. The table includes Swissmetro, NHTS 2022, four Biogeme public MNL datasets, and eleven R `mlogit` package datasets. Biogeme and Apollo are now run for every MNL row, while `mlogit`, `gmnl`, and `xlogit` are included

where aligned package wrappers are available. `xlogit` is especially fast on small standard MNL rows because the wrapper passes a prebuilt long-format numeric design matrix directly to a specialized in-process MNL solver; the comparison therefore separates this low-overhead standard-MNL path from broader package coverage such as symbolic specification, ragged choice sets, and non-MNL models. ModeCanada retains attempted failures for `gmnl` and `xlogit`; these failures are not treated as consistency failures for the successful aligned estimators.

Table 2: Real-data nested-logit-family runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for each aligned estimator. Consistency is evaluated against the aligned Biogeme implementation.

Data	Model	N	TorchDCM	Biogeme	Apollo	Consistent?
Swissmetro	Nested logit	10719	0.068	4.246	2.089	Yes
LPMC London	Nested logit	81086	0.325	22.918	22.769	Yes
NHTS 2022	Nested logit	27375	0.377	66.199	13.118	Yes
Parking Spain	Nested logit	1576	0.033	5.472	1.248	Yes
Airline itinerary	Nested logit	3609	0.089	6.364	1.255	Yes
Catsup	Nested logit	2798	0.071	8.782	1.209	Yes
Cracker	Nested logit	3292	0.054	8.988	1.256	Yes
Electricity	Nested logit	4308	0.118	18.301	1.386	Yes
Fishing	Nested logit	1182	0.095	7.558	1.148	Yes
HC	Nested logit	250	0.076	30.457	1.153	Yes
Heating	Nested logit	900	0.087	7.588	1.137	Yes
Mode	Nested logit	453	0.043	7.045	1.139	Yes

Table 2 reports the current real-data nested-logit-family benchmark. The table now includes a broader set of real-data cases for which we define an aligned nested-logit specification: Swissmetro, LPMC London, NHTS 2022, Parking Spain, Airline itinerary, and seven R `mlogit` datasets. The added R examples cover brand choice, product choice, electricity plan choice, recreational fishing, heating/cooling, home heating, and travel mode choice. The consistency flag is evaluated against Biogeme, which is the common nested-logit reference across these rows; Apollo runtimes are reported where the aligned wrapper completes, but Apollo is not used to determine the final consistency flag. All reported rows are consistent with Biogeme under the prespecified likelihood, parameter, and probability tolerances.

Table 3: Real-data mixed-logit full-estimation runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds. All cross-estimator rows use CPU for TorchDCM and Biogeme, 32 shared antithetic draws, a shared $\sigma = 1.0$ initial value, and 2–4 independent normal random coefficients selected from observed-variable coefficients where available. Exact utility specifications and random-coefficient names are recorded in the generated validation artifacts.

Data	N	RC	TorchDCM	Biogeme	Consistent?	Notes
Swissmetro	10719	2	0.217	16.627	Yes	–
Airline	3609	3	0.069	22.599	Yes	–
Parking Spain	1576	3	0.099	23.789	No	Different optimum
Telephone	434	2	0.032	20.069	No	Different optimum
LPMC London	81086	2	7.894	31.805	No	Different optimum
`mlogit::Car`	–	–	–	–	No	Skipped
`mlogit::Catsup`	2798	3	0.100	29.881	Yes	–
`mlogit::Cracker`	3292	3	0.118	262.630	No	Different optimum
`mlogit::Electricity`	4308	4	0.431	57.346	No	Non-finite diff
`mlogit::Fishing`	1182	2	0.077	19.435	No	Different optimum
`mlogit::Game`	–	–	–	–	No	Skipped
`mlogit::Game2`	–	–	–	–	No	Skipped
`mlogit::HC`	250	2	0.152	34.262	No	Different optimum
`mlogit::Heating`	900	2	0.149	23.032	Yes	–
`mlogit::JapaneseFDI`	–	–	–	–	No	Killed
`mlogit::Mode`	453	2	0.237	18.436	Yes	–
`mlogit::ModeCanada`	4324	4	38.682	54.785	Yes	–
`mlogit::Nox`	632	3	0.058	156.158	Yes	–
`mlogit::RiskyTransport`	1793	4	11.101	60.486	No	Different optimum
`mlogit::Train`	2929	4	0.332	30.663	No	Different optimum

Notes: "No" rows are retained as objective-diagnostic cases rather than treated as implementation failures. They indicate that TorchDCM and Biogeme finished the same simulated-likelihood specification at different final objectives or with non-finite replay diagnostics under the current optimizer settings. For completed "No" rows, Biogeme minus TorchDCM final simulated log-likelihood gaps are: Parking Spain -4.98×10^{-3}, Telephone -1.37×10^{-1}, LPMC London -1.14×10^{2}, `mlogit::Cracker` -2.20×10^{1}, `mlogit::Electricity` non-finite, `mlogit::Fishing` -5.62, `mlogit::HC` -6.37×10^{1}, `mlogit::RiskyTransport` -2.50, and `mlogit::Train` 1.34×10^{2}; negative-log-likelihood objective gaps have the opposite sign. The generated validation artifacts report each backend's final simulated log likelihood, parameter difference, probability difference, and full utility specification.

Table 3 reports the current real-data mixed-logit benchmark. Each runnable row uses 2–4 independent normal random coefficients, 32 shared antithetic draws, a shared $\sigma = 1.0$ initial value, and CPU execution for both TorchDCM and Biogeme. The battery includes Swissmetro, four Biogeme/LPMC public datasets, and fifteen `mlogit` package datasets; four rows are skipped because the exported data or the requested 2–4 random-coefficient specification is not numerically usable under the current harness. Seven full-estimation rows match Biogeme under the shared-draw consistency rule: Swissmetro, Airline, `mlogit::Catsup`, `mlogit::Heating`, `mlogit::Mode`, `mlogit::ModeCanada`, and `mlogit::Nox`. The remaining completed rows are retained as objective diagnostics rather than forced into agreement: both estimators solve the same simulated-likelihood specification, and the comparison records the final simulated log likelihood/objective reached by each optimizer. The generated validation artifacts record the exact utility specification, random-coefficient set, final objective values, likelihood differences, parameter differences, and probability

differences for each dataset.

Table 4: Benchmark case scale and alignment mode. The runtime table reports only estimator wall-clock time and consistency; this table records sample size, parameter count, and whether the row is full estimation or a shared-parameter replay.

Case	Model	N	K	Alignment mode
Swissmetro	MNL	10719	4	Full estimation with shared zero initial values.
Swissmetro	Nested logit	10719	5	Full estimation with shared zero initial values and common nest constraints.
Swissmetro	Cross-nested logit	10719	6	Full estimation with fixed allocation weights and common nest constraints.
Swissmetro	Mixed logit full estimation	10719	5	Full simulated maximum-likelihood estimation with 32 shared antithetic draws, observation-level likelihood, and shared $\sigma = 1.0$ initial value for TorchDCM and Biogeme.
Swissmetro	Mixed logit full estimation, 2 RC	10719	6	Full simulated maximum-likelihood estimation with independent normal random time and cost coefficients, 32 shared antithetic draws, and shared $\sigma = 1.0$ initial values for TorchDCM and Biogeme.
LPMC London	Nested logit	81086	7	Full estimation with active and motorized nests.
NHTS 2022	Nested logit	27375	18	Full estimation with active and motorized nests.
Parking Spain	Nested logit	1576	6	Full estimation with facility and pickup nests.
Airline itinerary	Nested logit	3609	6	Full estimation with two similar-itinerary alternatives nested together.
Catsup	Nested logit	2798	4	Full estimation with Heinz alternatives in a common brand nest.
Cracker	Nested logit	3292	4	Full estimation with national-brand alternatives in a common nest.
Electricity	Nested logit	4308	8	Full estimation with two tariff-plan nests.
Fishing	Nested logit	1182	4	Full estimation with shore and boat nests.
HC	Nested logit	250	4	Full estimation with current-system and new-system heating/cooling nests.
Heating	Nested logit	900	4	Full estimation with gas, electric, and heat-pump nests.
Mode	Nested logit	453	4	Full estimation with private and transit nests.
Swissmetro	Mixed logit replay	10719	5	Fixed-parameter likelihood/probability replay with 64 shared antithetic draws and panel likelihood.
Swissmetro	WTP mixed replay	10719	5	Fixed-parameter WTP-space likelihood/probability replay with 64 shared antithetic draws and panel likelihood.
Optima	Ordered logit	1822	9	Full estimation on the aligned Biogeme Optima ordered-response specification.
Optima	Ordered probit	1822	9	Full estimation on the aligned Biogeme Optima ordered-response specification.
Airline itinerary	MNL	3609	5	Full estimation on the Biogeme airline itinerary choice data.
Parking Spain	MNL	1576	5	Full estimation on the Biogeme parking choice data.
Telephone service	MNL	434	5	Full estimation on the Biogeme telephone-service choice data.
LPMC London	MNL	81086	5	Full estimation on the London Passenger Mode Choice data.
NHTS 2022	MNL	27375	16	Full estimation on a processed public-use trip-mode choice benchmark.

Continued on next page

Table 4: Benchmark case scale and alignment mode, continued.

Case	Model	N	K	Alignment mode
Fishing	MNL	1182	5	Full estimation against R `mlogit`, `gmnl`, and `xlogit`.
ModeCanada	MNL	4324	3	Full estimation against R `mlogit`.
Catsup	MNL	2798	3	Full estimation against R `mlogit`.
Cracker	MNL	3292	3	Full estimation against R `mlogit`.
Electricity	MNL	4308	6	Full estimation against R `mlogit`.
HC	MNL	250	2	Full estimation against R `mlogit`.
Heating	MNL	900	2	Full estimation against R `mlogit`.
Mode	MNL	453	2	Full estimation against R `mlogit`.
NOx	MNL	632	3	Full estimation against R `mlogit`.
RiskyTransport	MNL	1793	7	Full estimation against R `mlogit`.
Train	MNL	2929	4	Full estimation against R `mlogit`.

Table 4 records the corresponding parameter counts and alignment modes. The raw numerical-difference audit, including likelihood, parameter, probability, covariance, and standard-error differences, is retained in the generated validation artifacts rather than repeated in the paper-facing runtime tables.

Table 5: Pure synthetic controlled MNL runtime benchmarks. These cases are generated from a synthetic data-generating process and are not based on Swissmetro or any public empirical dataset. The benchmark varies sample size N, number of alternatives J, number of variables K, and feature correlation ρ, and reports total TorchDCM runtime.

Sweep	Case	N	J	K	ρ	Params	Total (s)	Consistent?
Sample size	$N = 1{,}000$	1000	4	6	0.30	9	0.021	Yes
Sample size	$N = 10{,}000$	10000	4	6	0.30	9	0.012	Yes
Sample size	$N = 100{,}000$	100000	4	6	0.30	9	0.068	Yes
Alternatives	$J = 3$	20000	3	6	0.30	8	0.014	Yes
Alternatives	$J = 20$	20000	20	6	0.30	25	0.409	Yes
Variables	$K = 4$	20000	5	4	0.30	8	0.021	Yes
Variables	$K = 32$	20000	5	32	0.30	36	0.113	Yes
Correlation	$\rho = 0.00$	20000	5	12	0.00	16	0.031	Yes
Correlation	$\rho = 0.98$	20000	5	12	0.98	16	0.054	Yes

Table 5 shows the pure synthetic scaling results using total runtime only. The synthetic cases are not derived from Swissmetro or any other public empirical dataset; covariates, utility parameters, and choices are generated directly from a synthetic MNL data-generating process. The sample-size sweep shows that TorchDCM remains fast as N grows: the $N = 100{,}000$ case has 400,000 long rows and completes in 0.068 seconds including covariance calculation. The variable-count sweep shows the effect of increasing the number of observed variables: the $K = 32$ case estimates 36 total parameters and completes in 0.113 seconds including covariance. The alternative-count sweep is also included because choice-set width is a central computational dimension for DCM estimators: the $J = 20$ case has 400,000 long rows and completes in 0.409 seconds including covariance. These synthetic experiments complement the public-data estimator comparisons by isolating controlled scaling behavior.

Table 6: Synthetic MNL full-estimation runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for aligned full estimation and covariance calculation where available. Rows vary sample size N, number of alternatives J, number of observed variables K, and equicorrelation ρ.

Case	N	J	K	ρ	TorchDCM	SciPy	Biogeme	Apollo	R packages	Consistent?
Base	1000	3	4	0.0	0.140	0.260	1.147	1.084	0.522/0.665/0.005	Yes
Large N	10000	3	4	0.0	0.012	9.685	0.808	1.769	0.733/1.381/0.035	Yes
Large J	3000	8	4	0.0	0.012	1.012	2.044	1.678	0.705/1.888/0.051	Yes
Large K	3000	4	12	0.0	0.009	2.364	2.219	1.946	0.655/2.048/0.053	Yes
High ρ	3000	4	6	0.8	0.007	1.061	1.531	1.326	0.601/1.218/0.023	Yes

Notes: the R package column reports `mlogit`/`gmnl`/`xlogit` total seconds. "Consistent" requires all successful external estimators to match TorchDCM under the same likelihood, parameter, and probability tolerances used for the real-data MNL benchmark.

Table 6 reports the synthetic MNL full-estimation comparisons. All rows use generated MNL data, shared zero initial values, and aligned generic long-format specifications for SciPy, Biogeme, Apollo, `mlogit`, `gmnl`, and `xlogit`. The five cases vary one design dimension at a time: baseline scale, larger N, larger J, larger K, and high feature correlation ρ. All successful estimators match TorchDCM under the same likelihood, parameter, and probability tolerances used for the real-data MNL benchmark.

Table 7: Synthetic nested-logit full-estimation runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for aligned full estimation and covariance calculation where available. Rows vary sample size N, number of alternatives J, number of observed variables K, and equicorrelation ρ.

Case	N	J	K	ρ	TorchDCM	Biogeme	Apollo	Consistent?
Base	1000	4	4	0.0	0.022	15.227	1.317	Yes
Large N	5000	4	4	0.0	0.056	15.245	1.930	Yes
Large J	2500	8	4	0.0	0.078	1031.681	1.468	Yes
Large K	2500	4	8	0.0	0.037	35.250	1.966	Yes
High ρ	2500	4	6	0.8	0.040	29.793	1.782	Yes

Notes: all rows use generated two-nest specifications with the same nested-logit structure across TorchDCM, Biogeme, and Apollo. Consistency is evaluated against Biogeme under the same likelihood, parameter, and probability tolerances used for the real-data nested-logit benchmark. The *Large J* Biogeme time includes substantial JAX/XLA CPU compilation and symbolic-estimation overhead observed in the remote run.

Table 7 reports the synthetic nested-logit comparisons. The five rows vary baseline scale, larger N, larger J, larger K, and high feature correlation ρ under generated two-nest specifications. As in the real-data NL table, consistency is evaluated against Biogeme; Apollo is included as an additional runtime and final-objective diagnostic. The generated nested-logit large-J row highlights a backend-overhead case: Biogeme remains numerically consistent but spends over 1000 seconds in total remote wall-clock time, including the observed JAX/XLA CPU compilation and symbolic-estimation overhead.

Table 8: Synthetic mixed-logit full-estimation runtime and consistency benchmarks. Runtime columns report total remote wall-clock seconds for aligned full estimation and covariance calculation where available. Rows vary sample size N, number of alternatives J, number of observed variables K, and equicorrelation ρ.

Case	N	J	K	ρ	RC	TorchDCM	Biogeme	Apollo	Consistent?
Base	1000	3	4	0.0	2	0.062	20.893	1.987	Yes
Large N	3000	3	4	0.0	2	0.092	19.721	4.318	Yes
Large J	1500	5	4	0.0	2	0.077	48.593	4.736	Yes
Large K	1500	4	8	0.0	4	0.170	60.486	8.612	Yes
High ρ	1500	4	6	0.8	3	0.084	54.087	5.606	Yes

Notes: rows use 2–4 independent normal random coefficients (RC) and 32 shared antithetic simulation draws. Consistency is evaluated against Biogeme under the same likelihood, parameter, and probability tolerances used for the real-data mixed-logit benchmark; Apollo is reported as an additional runtime and final-objective diagnostic.

Table 8 reports the synthetic mixed-logit comparisons. The five rows use 2–4 independent normal random coefficients with 32 shared antithetic simulation draws while varying N, J, K, and ρ. As in the real-data MixL table, consistency is evaluated against Biogeme; Apollo is included as an additional runtime and final-objective diagnostic. All reported synthetic mixed-logit rows are consistent with the Biogeme reference under the prespecified tolerances.

Table 9: Generated large-scale stress benchmarks with a 300-second external-backend timeout. Runtime columns report total remote wall-clock seconds when the backend completed, and "Timeout" when it did not finish within 300 seconds.

Family	N	J	K	ρ	RC	TorchDCM	Biogeme	Apollo	Status
MNL	50000	35	20	0.5	–	6.224	Timeout	Timeout	External timeout
Nested logit	50000	20	12	0.5	–	2.678	Timeout	57.066	Biogeme timeout
Mixed logit	20000	8	8	0.5	4	2.560	134.011	Timeout	Apollo timeout

Notes: stress rows are designed to identify scaling boundaries rather than numerical disagreement. TorchDCM completed all three full-estimation runs, while at least one external symbolic/reference backend exceeded the 300-second wall-clock budget. The MNL stress row is the strongest boundary case: both Biogeme and Apollo timed out while TorchDCM completed in 6.224 seconds.

Table 9 pushes the generated cases to larger N, J, and K values and imposes a 300-second wall-clock timeout on Biogeme and Apollo. These rows are stress tests rather than consistency tests: a timeout indicates that the external backend did not finish the aligned full-estimation run within the budget. TorchDCM completes all three stress cases in under seven seconds. In the largest MNL stress case, both Biogeme and Apollo exceed the 300-second timeout, while TorchDCM completes in 6.224 seconds.

Table 10: TorchDCM CPU/GPU mixed-logit device stress benchmarks. Runtime columns report remote wall-clock seconds for the TorchDCM model path only: device setup/compile, LBFGS simulated-likelihood estimation, and final log-likelihood replay. Synthetic data generation and Hessian covariance are excluded from this device-isolation experiment.

Case	N	J	K	ρ	RC	Draws	CPU	GPU	Status
MixL medium stress	50000	12	12	0.5	8	256	124.860	4.205	Consistent; 29.7×
MixL large stress	100000	12	12	0.5	8	512	Timeout	9.155	CPU timeout; GPU completed

Notes: both rows use the same generated mixed-logit specification, zero/shared initial values, and antithetic normal draws on CPU and CUDA. The CPU worker is capped at 300 seconds. On the completed 50k row, CPU and GPU estimates agree to 5.7×10^{-8} in final log likelihood and 1.9×10^{-7} in maximum parameter difference. The large row uses 25.1 GB peak CUDA memory on an NVIDIA GeForce RTX 5090 and implies a lower-bound model-path speedup above 32.8× relative to the 300-second CPU timeout.

Table 10 isolates the effect of the TorchDCM execution device on generated mixed-logit simulated-likelihood estimation. The same TorchDCM code path is run with `device='cpu'` and `device='cuda'` using identical initialization and draws. On the 50k-observation stress row, the GPU result is numerically indistinguishable from the CPU result while reducing model-path runtime from 124.860 seconds to 4.205 seconds. On the 100k-observation, 512-draw row, the CPU worker exceeds the 300-second wall-clock budget whereas the GPU completes the same model path in 9.155 seconds. This experiment is reported separately from the cross-estimator stress table because it targets hardware acceleration within TorchDCM rather than agreement with an external reference estimator.

Ordered response models are validated on the Biogeme Optima data. TorchDCM completes ordered logit full estimation in 0.042 seconds compared with 2.898 seconds for Biogeme. For ordered probit, TorchDCM completes estimation in 0.054 seconds compared with 3.475 seconds for Biogeme. Both ordered-response rows are marked consistent with the Biogeme reference implementation.

7 Discussion and Limitations

The current results support the main software claim that TorchDCM can combine PyTorch-native likelihood computation with econometric outputs and external estimator parity checks. The strongest evidence comes from full-estimation MNL and ordered-model cases, where the benchmark suite reports runtime splits and a prespecified consistency flag against reference estimators. Nested and cross-nested logit results extend this evidence to more structured substitution patterns, while the mixed-logit results now include real-data and generated full-estimation comparisons against Biogeme plus Apollo runtime diagnostics.

The main limitation is now breadth rather than existence of mixed-logit full estimation. The current full mixed-logit comparisons align TorchDCM and Biogeme under shared simulation draws and a common starting point, and they also report Apollo full-estimation runtime under Apollo's native Halton draw convention. The next experimental step is therefore to broaden full mixed-logit comparisons to additional public datasets and additional mixed-logit packages, while documenting draw conventions and optimizer initialization explicitly.

A second limitation is that covariance comparisons for constrained models require careful interpretation. For nested and cross-nested logit, different packages may report covariance on transformed or constrained parameter scales, and numerical differences can be larger than likelihood or probability differences. The manuscript should therefore avoid claiming universal covariance

equivalence until the transformation convention is audited and documented. The raw covariance and standard-error audits remain useful internally, but the paper-facing table should report only the final consistency flag.

The benchmark system is also still expanding in dataset coverage and controlled synthetic coverage. The current public data suite includes several Biogeme and R package examples, plus processed large-data candidates, and the current synthetic suite covers MNL, nested-logit, and mixed-logit full-estimation comparisons under controlled dimensions. The generated stress rows also identify a practical scaling boundary: at larger N, J, and K, TorchDCM continues to complete full estimation while at least one symbolic reference backend exceeds a 300-second wall-clock budget. More Apollo examples and additional public large-scale choice datasets should be added before final submission. This expansion will make the empirical section closer to IJOC software-paper standards, where reproducibility, public data, and baseline breadth are central acceptance signals.

8 Conclusion

This paper introduces TorchDCM, a PyTorch-first package for discrete choice model estimation, inference, and estimator benchmarking. The key idea is to pair tensor-native likelihood computation with econometric abstractions and a validation layer that compares against mature reference packages. Current public and generated benchmarks show full-estimation numerical parity for MNL, nested logit, cross-nested logit, ordered logit, ordered probit, and aligned mixed-logit cases against Biogeme, with Apollo included as an additional runtime and objective diagnostic where wrappers are available. Generated stress tests further show that TorchDCM can complete larger synthetic MNL, nested-logit, and mixed-logit estimations when Biogeme or Apollo exceed a 300-second wall-clock budget. The strongest immediate evidence is that TorchDCM matches reference estimators on likelihoods, parameters, probabilities, covariance outputs, and standard errors while often reducing core runtime by one to two orders of magnitude on the tested cases.

Future work will focus on broadening full mixed-logit estimation benchmarks across more public datasets and packages, aligning additional draw conventions across estimators, and expanding the public dataset suite. These extensions will strengthen the package as open infrastructure for researchers who need differentiable choice models, econometric inference, and reproducible cross-estimator validation.

Reproducibility Statement

The TorchDCM repository separates installable package code from heavyweight validation. Package code is stored under `torchdcm/`, public small datasets under `datasets/small/`, documentation under `docs/`, and benchmark wrappers and generated results under `validation/`. The paper-facing solver matrix can be run on the remote benchmark machine with:

```
cd /home/baichuan-mo/torchdcm
.venv/bin/python validation/benchmarks/run_solver_attempt_matrix.py \
  --profile full \
  --python /home/baichuan-mo/torchdcm/.venv/bin/python
```

The paper-facing real-data benchmark tables are constructed from the remote solver-attempt matrix in `validation/generated/solver_attempt_matrix_full.json`, with detailed case-level audits retained in the generated validation files. Table-specific case scale and alignment details are

recorded in `paper/tables/benchmark_cases.tex`. The broader Markdown benchmark summary is maintained in `docs/model-family-benchmark-comparison.md`.

References

[1] Camilo Arteaga, Jinwoo Park, and Alexander Paz. xlogit: An open-source Python package for GPU-accelerated estimation of mixed logit models. *Journal of Choice Modelling*, 42:100339, 2022.

[2] Michel Bierlaire. A short introduction to biogeme. Technical Report TRANSP-OR 230620, Transport and Mobility Laboratory, EPFL, 2023. URL https://biogeme.epfl.ch/.

[3] Yves Croissant. Estimation of random utility models in R: The mlogit package. *Journal of Statistical Software*, 95(11):1–41, 2020.

[4] Stephane Hess and David Palma. Apollo: A flexible, powerful and customisable freeware package for choice model estimation and application. *Journal of Choice Modelling*, 32:100170, 2019.

[5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[6] Mauricio Sarrias and Ricardo A. Daziano. Multinomial logit models with continuous and discrete individual heterogeneity in R: The gmnl package. *Journal of Statistical Software*, 79(2): 1–46, 2017.

A Claim-Evidence Map

Claim	Evidence in Current Draft	Status
TorchDCM is PyTorch-first and exposes discrete-choice abstractions.	Package design section describes `ChoiceDataset`, `UtilitySpec`, `Beta`, model classes, and result objects; repository structure supports this.	Supported
TorchDCM matches reference estimators for MNL full estimation.	Swissmetro, public Biogeme MNL battery, Fishing, and ModeCanada benchmark rows are marked consistent under the prespecified tolerance rule.	Supported
TorchDCM matches reference estimators for nested and cross-nested logit.	Swissmetro nested and cross-nested full-estimation rows are marked consistent under the prespecified tolerance rule.	Supported
TorchDCM validates mixed-logit kernels.	Shared-draw fixed replay against Biogeme and Apollo gives machine-precision probability and likelihood agreement.	Supported
TorchDCM has an initial full mixed-logit estimator parity result.	Swissmetro mixed logit with a normal random time coefficient matches Biogeme under shared draws and a shared $\sigma = 1.0$ initial value; Apollo full-estimation runtime is also reported under Apollo's native Halton draw convention.	Supported with scope limit
TorchDCM provides faster runtime on current benchmark cases.	Tables report lower TorchDCM total or estimation time than Biogeme/Apollo/mlogit in most econometric-reference cases, with `xlogit` faster on Fishing MNL.	Supported with nuance
TorchDCM scales on pure synthetic MNL data.	Synthetic benchmark table reports runtime splits as N, J, K, and feature correlation vary.	Supported for MNL

B Reviewer-Facing Self-Review

Dimension	Reviewer-facing question	Current status
Contribution	Does the paper provide new software infrastructure rather than only a reimplementation?	Pass; clarify PyTorch-first plus benchmark-system contribution.
Writing clarity	Can a reader reproduce the package workflow and benchmark protocol?	Partial; add more API examples and exact environment details before submission.
Experimental strength	Are strong baselines included?	Partial; Biogeme, Apollo, SciPy, mlogit, gmnl, and xlogit are included, and mixed-logit full estimation is now aligned against Biogeme with Apollo runtime reported under its native draw convention.
Evaluation completeness	Are all major model-family claims supported by full estimation?	Improved; standard mixed logit has one full-estimation row, while WTP mixed still uses fixed replay.
Method soundness	Are hidden assumptions documented?	Partial; add constrained-parameter covariance convention and draw-generation details.